



Centro Universitário de Excelência · Sistemas de Informação

MecaniQA API

Gestão de Peças e Serviços

OAT 1 · Desenvolvimento Web Orientado a Objetos

Equipe Palmas

Arthur Machado Morais
Felipe Souza Teixeira da Silva
Ian da Silva Borges
Jonatan de Souza Ferreira
Victor Tadeu Araújo Augusto

Itabuna, 2026



Roteiro

01 Modelagem

Peça, Serviço e CategoriaPeca.

02 Decisão do enum

Categoria controlada, sem texto livre.

03 Singleton

Repositories manuais em memória.

04 Controllers

Integração sem injeção do Spring.

05 Rotas REST

Endpoints e status HTTP.

06 Artefatos

UML, README e Collection.

Modelagem dos dados

A base da OAT ficou em duas entidades principais e uma enumeração para organizar o catálogo.

classe

Peca

- codigo gerado pela aplicação
- codigoBarras e fornecedor/marca
- quantidadeEstoque
- precoCusto e precoVenda
- dataCadastro e dataUltimaAtualizacao
- tamanho e cor opcionais
- categoria obrigatória

enum

CategoriaPeca

- MOTOR
- SUSPENSAO
- FREIOS
- ELETRICA
- ACESSORIOS

classe

Servico

- codigo gerado pela aplicação
- nome do serviço
- tempoEstimadoMinutos
- custoTabelado
- dataCriacao e dataUltimaAtualizacao

Por que usar CategoriaPeca?

A enumeração foi a forma mais simples de manter o catálogo padronizado.

Sem enum

- Categoria escrita livremente.
- Risco de grafias diferentes.
- Mais difícil filtrar e validar.

CategoriaPeca

MOTOR
SUSPENSAO
FREIOS
ELETRICA
ACESSORIOS

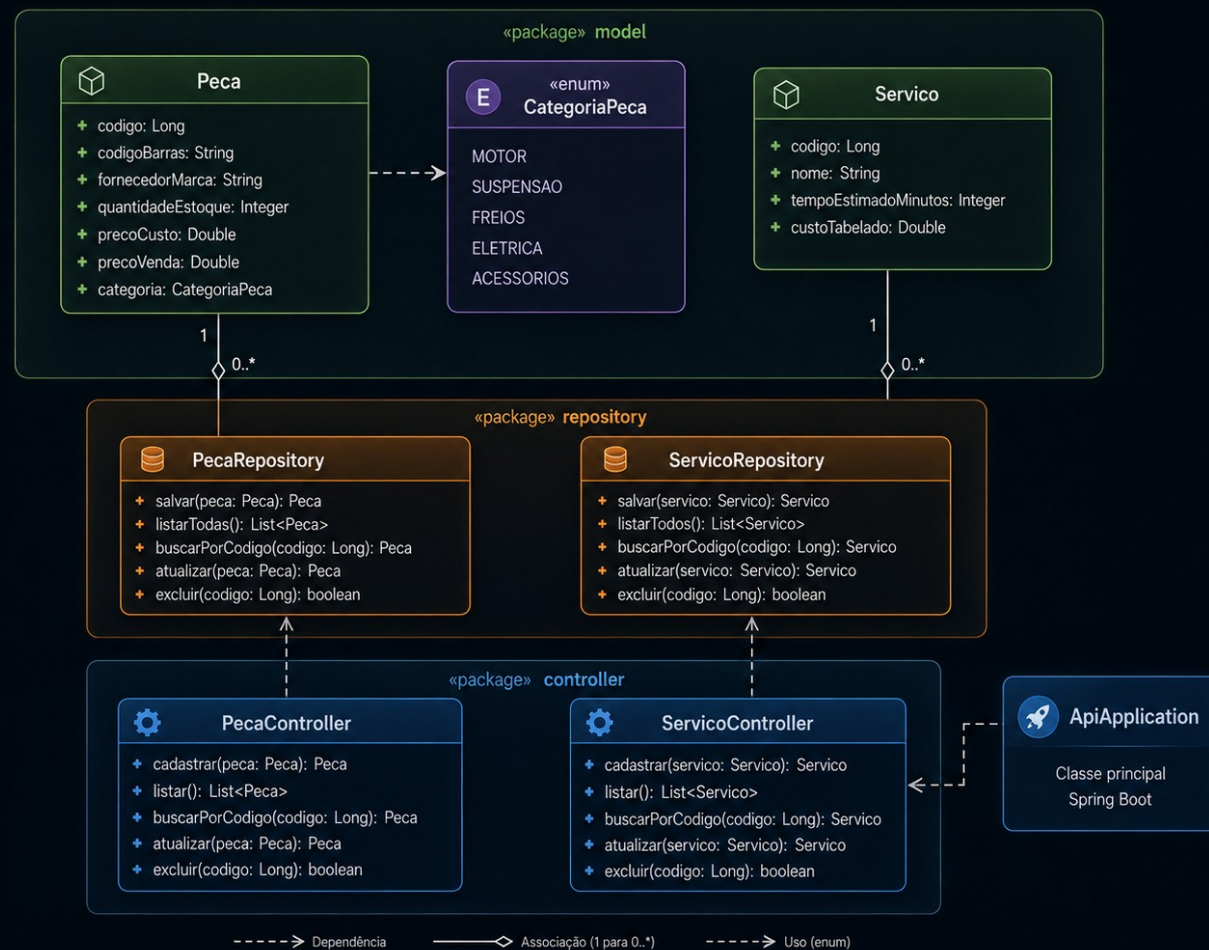
Com enum

- Valores fechados e previsíveis.
- Validação mais direta no cadastro.
- Diagrama mostra o vínculo com Peça.

Diagrama de Classes

A visualização junta entidades, enum, repositories, controllers e relações principais.

Diagrama de Classes – MecaniQA API (OAT 1)



Por que escolhemos Singleton?

A OAT não permitia banco de dados, Spring Data ou injeção de dependência. O repository precisava ser manual.

private

Construtor fechado

Evita que alguém crie outro repository com new.

static

Instância única

Mantém o mesmo objeto acessível durante a execução.

getInstance()

Ponto de acesso

Controller pede a instância sem usar @Autowired.

List<T>

Memória simples

As peças e serviços ficam nas listas internas.

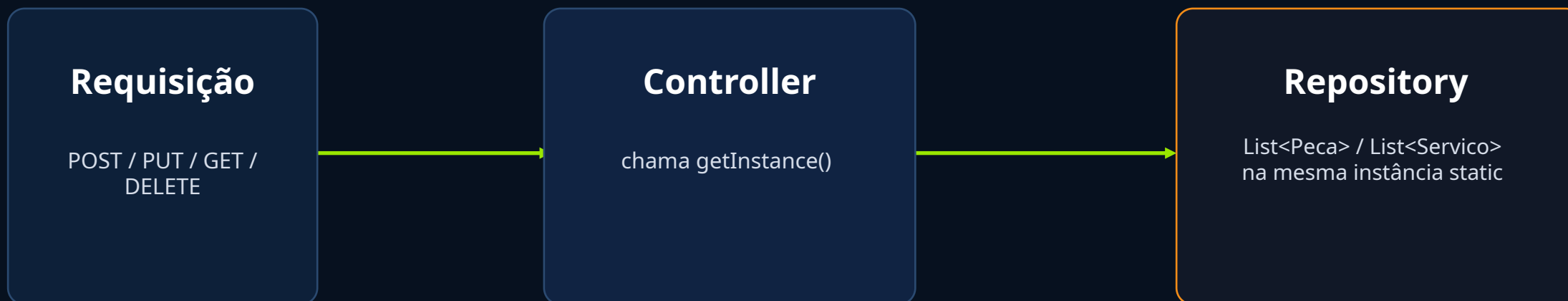
Controller

getInstance()

Repository Singleton

Como os dados se mantêm enquanto a API roda?

Como não existe banco nesta fase, as listas internas cumprem o papel de armazenamento temporário.



Quando o servidor para, os dados em memória são perdidos. Isso é esperado nesta etapa, porque a OAT foca em OO e estrutura da API.

Controller sem injeção do Spring

A integração foi feita chamando o Singleton manualmente dentro de cada controller.

PecaController

```
private final PecaRepository repository =  
    PecaRepository.getInstance();
```

```
@PostMapping
```

```
cadastrar(@RequestBody Peca peca)
```

ServicoController

```
private final ServicoRepository repository =  
    ServicoRepository.getInstance();
```

```
@GetMapping
```

```
listar() / buscar(codigo)
```

A escolha reforça o requisito do guia: repository manual, sem @Autowired e sem Spring Data nesta primeira versão.

Rotas REST e respostas semânticas

A API não devolve apenas dados: cada operação retorna o status HTTP esperado.

Peças

POST /api/pecas

GET /api/pecas

GET /api/pecas/1

PUT /api/pecas/1

DELETE /api/pecas/1

Serviços

POST /api/servicos

GET /api/servicos

GET /api/servicos/1

PUT /api/servicos/1

DELETE /api/servicos/1

201 Created

200 OK

204 No Content

400 Bad Request

404 Not Found

Pontos validados pelo QA

Os testes foram pensados a partir das perguntas do guia: enum, Singleton e respostas HTTP.

Enum obrigatório

Peça só é cadastrada com categoria válida em CategoriaPeca.

Construtor privado

Não é possível instanciar PecaRepository com new fora da classe.

Cadastro em memória

POST grava na lista interna e retorna 201 Created.

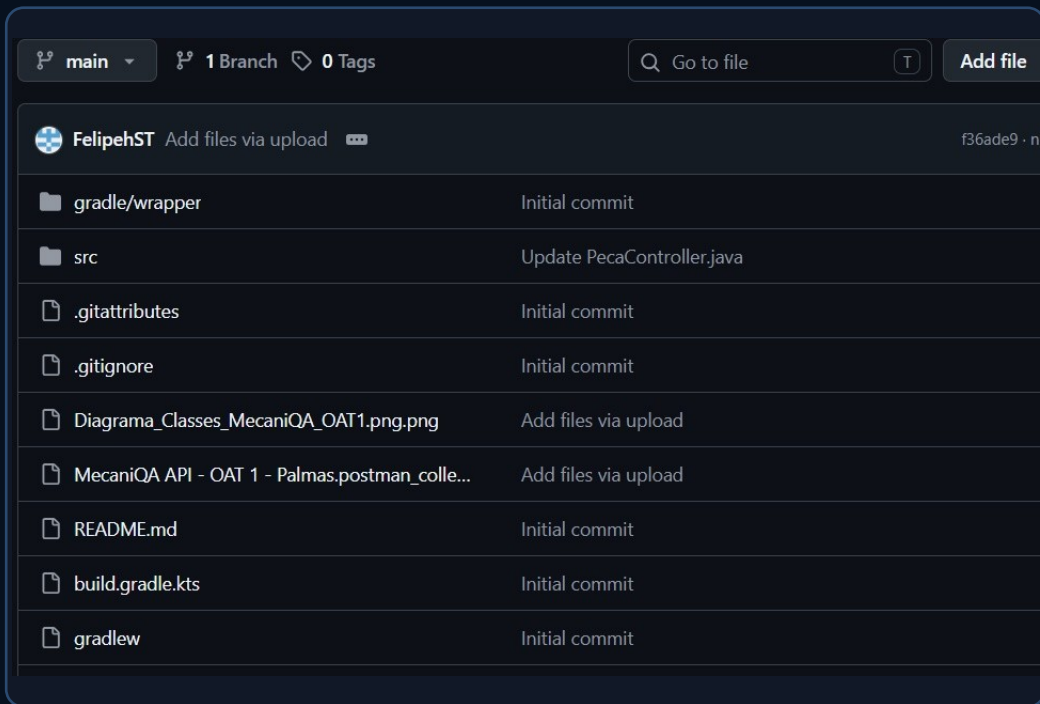
Busca inexistente

Quando o ID não existe, a API retorna 404 Not Found.

A Collection do Postman ficou como apoio para repetir os cenários durante a avaliação.

Artefatos da entrega

Além do código, deixamos materiais que ajudam o professor a entender e testar a solução.



main	1 Branch	0 Tags	Go to file	Add file
FelipehST Add files via upload f36ade9 · n				
gradle/wrapper	Initial commit			
src	Update PecaController.java			
.gitattributes	Initial commit			
.gitignore	Initial commit			
Diagrama_Classes_MecaniQA_OAT1.png.png	Add files via upload			
MecaniQA API - OAT 1 - Palmas.postman_colle...	Add files via upload			
README.md	Initial commit			
build.gradle.kts	Initial commit			
gradlew	Initial commit			

Código Java

Models, repositories e controllers implementados.

Diagrama UML

Classes, enum, Singleton e relações principais.

Collection

Rotas prontas para importação no Postman.

README

Resumo do projeto e comandos de execução.

A entrega fica pronta para ser aberta, executada e conferida apenas pelo professor.

Referências usadas

OAT 1 enunciado com entidades, repositories, controllers e status HTTP.

Guia da OAT 1 perguntas de apoio usadas para justificar as escolhas.

Spring Boot estrutura da aplicação e anotações REST.

Postman apoio para testar as rotas criadas.